

AI ASSISTED CODING LAB TEST-3

NAME : MD.SANIYA

HT NO:2403A510E7

BATCH:05

SET:E7

QUESTION 1:

Prompt:

Scenario: In the E-commerce sector, a company faces a challenge related to algorithms with AI assistance.

Task: Use AI-assisted tools to solve a problem involving algorithms with AI assistance in this context.

Deliverables:

Submit the source code of your implemented solution.

Provide an explanation of the AI assistance used during the development process.

Include a sample output demonstrating how your algorithm solves the problem.

CODE- GENERATOR:

```
C: > Users > Anjali Jeeluka > lab_test3.py > evaluate_leave_one_out
1  """
2  Hybrid recommender combining content-based (TF-IDF on product des
3  and collaborative filtering (item-item similarity from user ratin
4
5  Deliverables in this file:
6  - build_matrices, compute_similarities, score_items_for_user, rec
7  - evaluate_leave_one_out (RMSE, Precision@K)
8  - sample data and demonstration run in __main__
9  Dependencies: numpy, pandas, scikit-learn
10 """
11 import numpy as np
12 import pandas as pd
13 from sklearn.feature_extraction.text import TfidfVectorizer
14 from sklearn.metrics.pairwise import cosine_similarity
15 from math import sqrt
16
17 def build_matrices(products_df, ratings_df, description_col="desc
18     """Build TF-IDF content matrix (items x features) and user-it
19     tfidf = TfidfVectorizer(stop_words="english")
20     content_matrix = tfidf.fit_transform(products_df[description_
21     user_item = ratings_df.pivot_table(index="user_id", columns="
22     return content_matrix, user_item
23
24 def compute_similarities(content_matrix, user_item):
25     """Compute item-item content similarity and collaborative sim
26     content_sim = cosine_similarity(content_matrix) # items x it
27     item_user = user_item.T.values # items x users
28     collab_sim = cosine_similarity(item_user) if item_user.size e
29     return content_sim, collab_sim
30
31 def score_items_for_user(user_id, products_df, user_item, content
32     """
33     Compute hybrid score for all items for a given user.
```

```

34     alpha: weight for content (0..1). (1-alpha) weight for collab
35     Returns DataFrame: product_id, score
36     """
37     all_items = list(user_item.columns)
38     n = len(all_items)
39     # If user unknown or has no ratings -> fall back to popularity
40     if user_id not in user_item.index or (user_item.loc[user_id].
41         pop = user_item.sum(axis=0)
42         return pd.DataFrame({"product_id": all_items, "score": po
43
44     user_ratings = user_item.loc[user_id].values # aligned with
45     # content score (for item j): sum_i content_sim[j,i] * rating
46     content_score = content_sim.dot(user_ratings)
47     denom_c = np.maximum(content_sim.sum(axis=1), 1e-8)
48     content_score = content_score / denom_c
49     # collaborative score
50     collab_score = collab_sim.dot(user_ratings)
51     denom_k = np.maximum(collab_sim.sum(axis=1), 1e-8)
52     collab_score = collab_score / denom_k
53     hybrid = alpha * content_score + (1 - alpha) * collab_score
54     return pd.DataFrame({"product_id": all_items, "score": hybrid
55
56 def recommend_for_user(user_id, products_df, user_item, content_s
57     """Return top_k recommendations for user_id (excludes already
58     scores = score_items_for_user(user_id, products_df, user_item
59     # exclude already rated
60     if user_id in user_item.index:
61         rated = user_item.loc[user_id] > 0
62         rated_items = set(user_item.columns[rated.values])
63         scores = scores[~scores["product_id"].isin(rated_items)]
64     merged = scores.merge(products_df[["product_id", "name", "pri
65     return merged.sort_values("score", ascending=False).head(top_

```

Click to add a breakpoint

```
67 def evaluate_leave_one_out(products_df, ratings_df, alpha=0.6, k=
68     """
69     Leave-one-out: for each user with >=2 ratings, hide one rating
70     predict it and compute RMSE and Precision@K across users.
71     """
72     rmses = []
73     hits = []
74     content_matrix, full_user_item = build_matrices(products_df,
75     content_sim_full, collab_sim_full = compute_similarities(content_ma
76     users = ratings_df["user_id"].unique()
77     for u in users:
78         ur = ratings_df[ratings_df["user_id"] == u]
79         if len(ur) < 2:
80             continue
81         test = ur.iloc[-1]
82         train = ratings_df.drop(test.name)
83         _, train_user_item = build_matrices(products_df, train)
84         content_sim, collab_sim = compute_similarities(content_ma
85         scores_df = score_items_for_user(u, products_df, train_us
86         # predicted score for the test product
87         row = scores_df[scores_df["product_id"] == test["product_
88         pred_score = row["score"].values[0] if not row.empty else
89         # map score to rating scale using min/max of scores_df ->
90         rmin, rmax = train["rating"].min(), train["rating"].max()
91         smin, smax = scores_df["score"].min(), scores_df["score"]
92         if smax == smin:
93             pred_rating = (rmin + rmax) / 2 if not np.isnan(rmin)
94         else:
95             pred_rating = rmin + (pred_score - smin) * (rmax - r
96         rmses.append((pred_rating - test["rating"]) ** 2)
97         # Precision@K
98         recs = recommend_for_user(u, products_df, train_user_item
```

```

100     rmse = sqrt(np.mean(rmses)) if rmses else None
101     prec_at_k = np.mean(hits) if hits else None
102     return rmse, prec_at_k
103
104 if __name__ == "__main__":
105     # Small sample dataset for demonstration
106     products = [
107         {"product_id": "A", "name": "Wireless Mouse",
108          "description": "ergonomic wireless mouse with optical sensor"},
109         {"product_id": "B", "name": "Gaming Keyboard",
110          "description": "mechanical keyboard RGB backlight for gaming"},
111         {"product_id": "C", "name": "USB-C Cable",
112          "description": "durable USB-C to USB-A cable 1m", "price": 15},
113         {"product_id": "D", "name": "Noise Cancelling Headphones",
114          "description": "over-ear headphones with active noise cancellation"},
115         {"product_id": "E", "name": "Laptop Stand",
116          "description": "adjustable aluminum laptop stand for ergonomic use"}
117     ]
118     ratings = [
119         {"user_id": 1, "product_id": "A", "rating": 5},
120         {"user_id": 1, "product_id": "B", "rating": 4},
121         {"user_id": 1, "product_id": "C", "rating": 2},
122         {"user_id": 2, "product_id": "A", "rating": 4},
123         {"user_id": 2, "product_id": "D", "rating": 5},
124         {"user_id": 3, "product_id": "B", "rating": 3},
125         {"user_id": 3, "product_id": "E", "rating": 4},
126         {"user_id": 4, "product_id": "C", "rating": 4},
127         {"user_id": 4, "product_id": "E", "rating": 5},
128     ]
129     products_df = pd.DataFrame(products)
130     ratings_df = pd.DataFrame(ratings)
131

```

```
C:\Users\Anjali Jeeluka> lab_test3.py > evaluate_leave_one_out
124     {"user_id": 3, "product_id": "B", "rating": 3},
125     {"user_id": 3, "product_id": "E", "rating": 4},
126     {"user_id": 4, "product_id": "C", "rating": 4},
127     {"user_id": 4, "product_id": "E", "rating": 5},
128 ]
129 products_df = pd.DataFrame(products)
130 ratings_df = pd.DataFrame(ratings)
131
132 content_matrix, user_item = build_matrices(products_df, ratings_df)
133 content_sim, collab_sim = compute_similarities(content_matrix)
134
135 # Show recommendations for some users
136 for uid in [1, 2, 3, 5]:
137     recs = recommend_for_user(uid, products_df, user_item, content_sim)
138     print(f"\nRecommendations for user {uid}:")
139     print(recs[["product_id", "name", "score"]].to_string(index=False))
140
141 # Evaluation
142 rmse, prec_at_3 = evaluate_leave_one_out(products_df, ratings_df)
143 print(f"\nEvaluation (leave-one-out): RMSE = {rmse:.4f} Prec@3 = {prec_at_3:.4f}")
```

Output:

```
PS C:\Users\Anjali Jeeluka\.vscode> & "C:/Users/Anjali Jeeluka/anaconda3/python.exe" "c:/Users/Anjali Jeeluka/lab_test3.py"

Recommendations for user 1:
product_id      name      score
E      Laptop Stand 0.896753
D Noise Cancelling Headphones 0.769000

Recommendations for user 2:
product_id      name      score
B Gaming Keyboard 0.424010
E      Laptop Stand 0.270393
C      USB-C Cable 0.232286

Recommendations for user 3:
product_id      name      score
C      USB-C Cable 0.643054
A      Wireless Mouse 0.558869
D Noise Cancelling Headphones 0.000000

Recommendations for user 5:
product_id      name      score
A Wireless Mouse      9.0
E      Laptop Stand      9.0
B Gaming Keyboard      7.0
```

Observation:

The hybrid recommender system combines content-based filtering (TF-IDF) and collaborative filtering to give personalized product suggestions in ecommerce. It predicts user preferences effectively, showing good accuracy (RMSE ≈ 0.79 , Precision@3 ≈ 0.67). AI tools helped in generating and optimizing the algorithm quickly. Overall, it improves recommendation quality and user satisfaction.

QUESTION 2:

PROMPT:

Scenario: In the Retail sector, a company faces a challenge related to data structures with AI. Task: Use AI-assisted tools to solve a problem involving data structures with AI in this context. Deliverables: Submit the source code of your implemented solution. Provide an explanation of the AI assistance used during the development process. Include a sample output demonstrating how your algorithm solves the problem.

CODE-GENERATED:

```
C:\Users > Anjali Jeeluka > que2.py > ...
1  """
2  Retail Data-Structures + AI demo
3
4  Features:
5  - Inverted index for fast keyword search across product attribute
6  - TF-IDF + cosine similarity for content-based "similar product"
7  - Min-heap based top-K selection for efficient retrieval.
8  - Low-stock alerts using a min-heap (priority by quantity).
9
10 Dependencies:
11 - scikit-learn
12 Standard libs: collections, heapq, typing
13 """
14 from collections import defaultdict
15 from typing import List, Dict, Tuple
16 import heapq
17
18 from sklearn.feature_extraction.text import TfidfVectorizer
19 from sklearn.metrics.pairwise import cosine_similarity
20
21
22 Product = Dict[str, object]
23
24
25 def build_inverted_index(products: List[Product], fields: List[str])
26     """
27     Build a simple token -> set(product_id) inverted index over s
28     Tokens are lower-cased words split on whitespace and punctuat
29     """
30     index = defaultdict(set)
31     for p in products:
32         pid = p["product_id"]
33         for f in fields:
```

```

34         text = str(p.get(f, "")).lower()
35         for token in text.split():
36             index[token].add(pid)
37     return index
38
39
40 def search_products(query: str, index: Dict[str, set], products_m
41     """
42     Perform AND search over query tokens using the inverted index
43     Returns products in original insertion order where possible.
44     """
45     tokens = [t.lower() for t in query.split() if t]
46     if not tokens:
47         return []
48     sets = [index.get(t, set()) for t in tokens]
49     matched_ids = set.intersection(*sets) if sets else set()
50     # preserve insertion order by products_map keys
51     return [products_map[pid] for pid in products_map if pid in m
52
53
54 def build_tfidf(products: List[Product], text_field: str = "descr
55     """
56     Build TF-IDF matrix and mappings for products.
57     """
58     docs = [p.get(text_field, "") for p in products]
59     ids = [p["product_id"] for p in products]
60     vec = TfidfVectorizer(stop_words="english")
61     tfidf = vec.fit_transform(docs)
62     id_to_idx = {pid: i for i, pid in enumerate(ids)}
63     idx_to_id = {i: pid for pid, i in id_to_idx.items()}
64     return tfidf, id_to_idx, idx_to_id
65

```

Click to add a breakpoint

```
similar(product_id: str, tfidf_matrix, id_to_idx: Dict[str, int],
        idx_to_id: Dict[int, str], products_map: Dict[int, Product],
        top_k: int = 5) -> List[Tuple[Product, float]]:
    """
    Recommend top_k products similar to given product_id using cosine similarity.
    Uses heapq.nlargest on (score, idx) to be memory/time efficient.
    """
    if product_id not in id_to_idx:
        return []
    i = id_to_idx[product_id]
    # compute similarity of item i to all items
    sims = cosine_similarity(tfidf_matrix[i], tfidf_matrix).ravel()
    # exclude self
    sims[i] = -1.0
    # get top_k indices
    if top_k <= 0:
        return []
    topk = heapq.nlargest(top_k, range(len(sims)), key=lambda x: sims[x])
    results = [(products_map[idx_to_id[j]], float(sims[j])) for j in topk]
    return results

def low_stock_alerts(products: List[Product], k: int = 5) -> List[Product]:
    """
    Return k products with lowest quantity (priority queue approach).
    """
    if k <= 0:
        return []
    # use nsmallest by quantity
    return heapq.nsmallest(k, products, key=lambda p: p.get("quantity"))
```

```

99 def demo():
100     # sample product catalog
101     products = [
102         {"product_id": "P1", "name": "Wireless Mouse", "description": "A wireless mouse with a long battery life."},
103         {"product_id": "P2", "name": "Gaming Keyboard", "description": "A mechanical gaming keyboard with RGB lighting."},
104         {"product_id": "P3", "name": "USB-C Cable", "description": "A high-speed USB-C cable for charging and data transfer."},
105         {"product_id": "P4", "name": "Noise Cancelling Headphones", "description": "Over-ear headphones with active noise cancellation."},
106         {"product_id": "P5", "name": "Laptop Stand", "description": "An adjustable laptop stand to improve ergonomics."},
107         {"product_id": "P6", "name": "Wireless Keyboard", "description": "A compact wireless keyboard with a long battery life."}
108     ]
109     # convenience map preserving insertion order (Python 3.7+)
110     products_map = {p["product_id"]: p for p in products}
111
112     # Build inverted index on name + description
113     index = build_inverted_index(products, ["name", "description"])
114
115     # Example: keyword search
116     query = "wireless keyboard"
117     search_res = search_products(query, index, products_map)
118     print("Search results for query:", query)
119     for p in search_res:
120         print(f" - {p['product_id']}: {p['name']} (qty={p['quantity']})")
121     print()
122
123     # Build TF-IDF and get similar recommendations
124     tfidf, id_to_idx, idx_to_id = build_tfidf(products, text_fields=["name", "description"])
125     pid = "P2" # Gaming Keyboard
126     recs = recommend_similar(pid, tfidf, id_to_idx, idx_to_id, products_map)
127     print(f"Top recommendations similar to {pid} ({products_map[pid]['name']})")
128     for prod, score in recs:
129         print(f" - {prod['product_id']}: {prod['name']} (score={score})")
130     print()

```

C: > Users > Anjali Jeeluka >  que2.py > ...

```

99 def demo():
127     print(f"Top recommendations similar to {pid} ({products_map[pid]['name']})")
128     for prod, score in recs:
129         print(f" - {prod['product_id']}: {prod['name']} (score={score})")
130     print()
131
132     # Low-stock alerts
133     alerts = low_stock_alerts(products, k=3)
134     print("Low stock alerts (top 3 lowest quantity):")
135     for p in alerts:
136         print(f" - {p['product_id']}: {p['name']} (qty={p['quantity']})")
137
138
139 if __name__ == "__main__":
140     demo()

```

Output:

```
conda3/python.exe" "c:/Users/Anjali Jeeluka/que2.py"
Search results for query: wireless keyboard
- P6: Wireless Keyboard (qty=0)

Top recommendations similar to P2 (Gaming Keyboard):
- P6: Wireless Keyboard (score=0.192)

Low stock alerts (top 3 lowest quantity):
- P6: Wireless Keyboard (qty=0)
- P4: Noise Cancelling Headphones (qty=5)
- P5: Laptop Stand (qty=8)
PS C:\Users\Anjali Jeeluka\.vscode>
```

Observation:

- The AI-assisted TF-IDF algorithm correctly finds similar products (e.g., “Wireless Keyboard” is similar to “Gaming Keyboard”).
- The inverted index enables fast keyword-based search.
- The min-heap efficiently identifies low-stock products for restocking alerts.